Foundations of Artificial Intelligence - 2026.1

Sequential Decisions under Uncertainty - Online Policy Improvement

Felix Mohr

Universidad de La Sabana

Chía, May 4, 2026

Decision Making Situations

Decision Situation Matrix

Action Effects

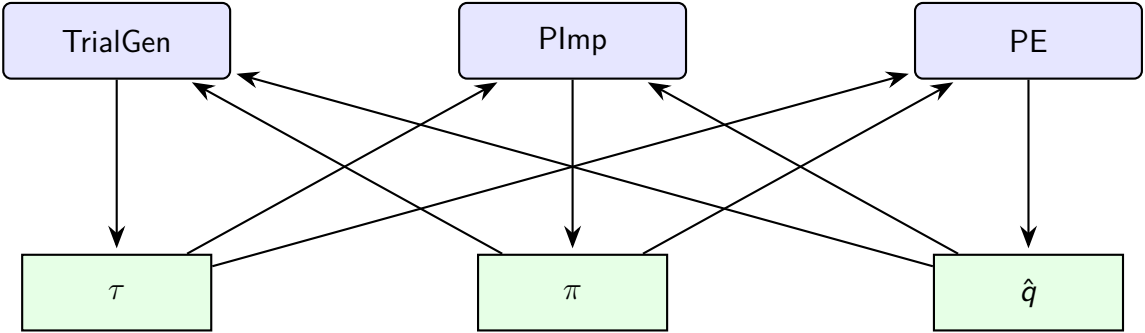
Horizon	**Certainty**	**Risk**	**Uncertainty**
Atomic	✓	✓	✓
Sequential	✓	✓	NOW (66%)

Class Overview

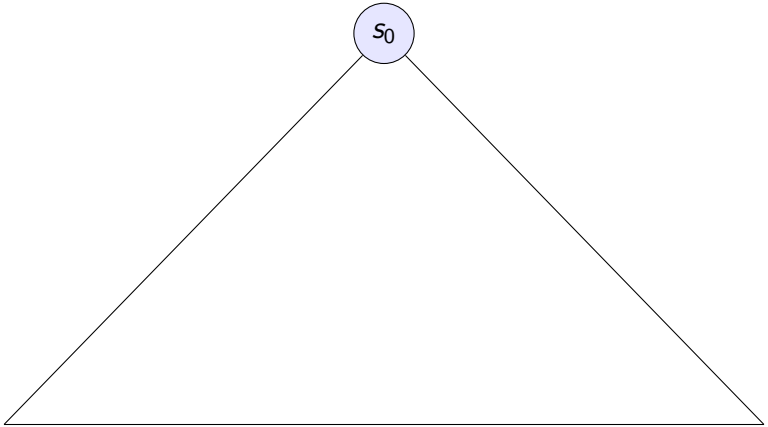
1. Motivation for Advanced Policy Improvement
2. Trial-Based Online Policy Improvement
3. Monte-Carlo Tree Search (MCTS)

Motivation for Advanced Policy Improvement

GPI With Centralized Trial Generator

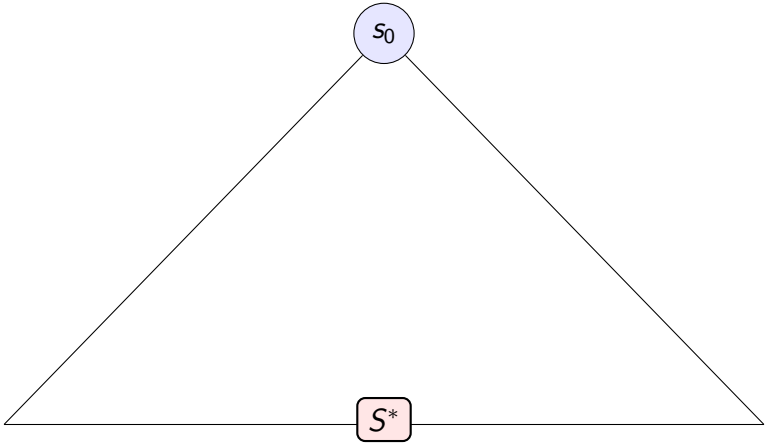


Online Policy Improvement



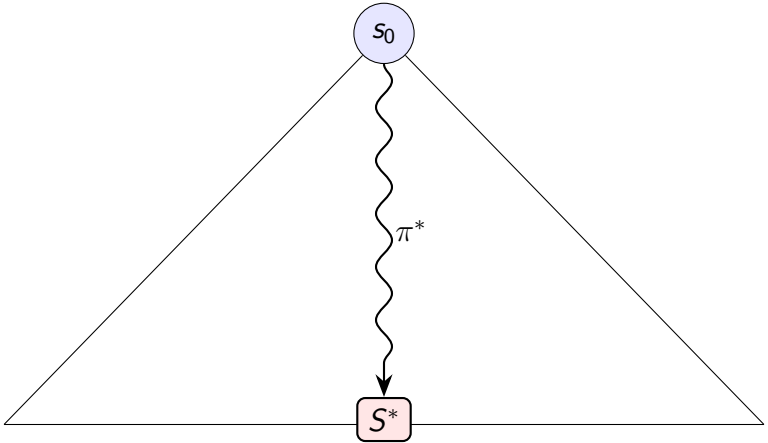
Online Policy Improvement

There is a small set S^* of desirable (and realistic) final states.



Online Policy Improvement

There is a small set S^* of desirable (and realistic) final states. An optimal policy π^* tries to make sure we end up in that region.



Online Policy Improvement

Ideally, we could create trial with such a policy π^*.

Then we could estimate $q(s, a)$ for any pair (s, a).

Online Policy Improvement

Ideally, we could create trial with such a policy π^*.

Then we could estimate $q(s, a)$ for any pair (s, a).

Of course if we had access to π^* the problem would already be solved.

Online Policy Improvement

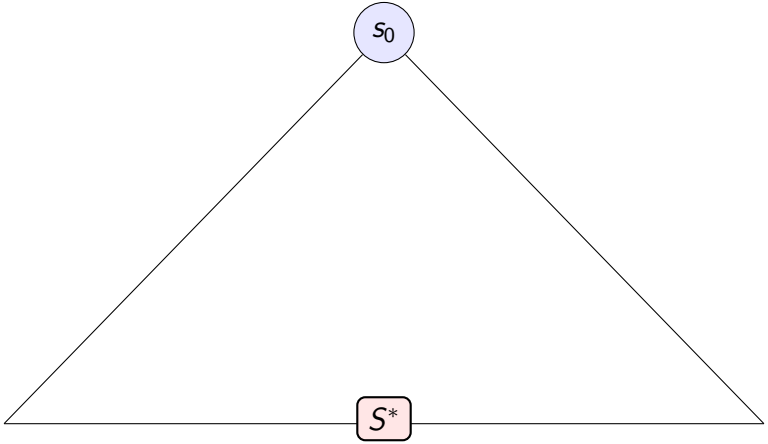
Ideally, we could create trial with such a policy π^*.

Then we could estimate $q(s, a)$ for any pair (s, a).

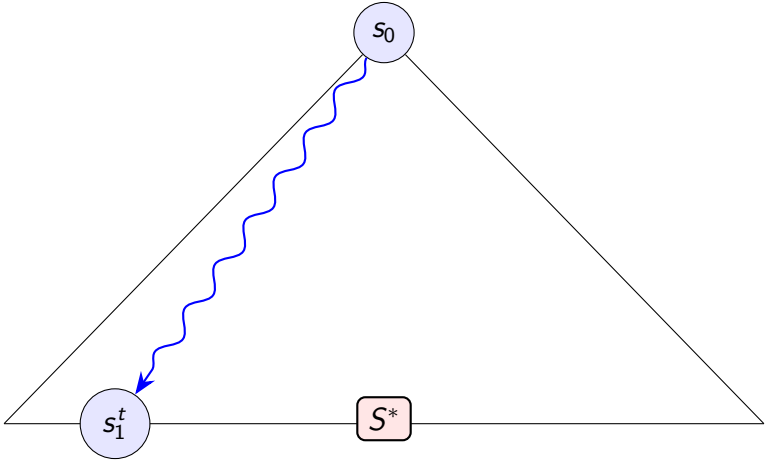
Of course if we had access to π^* the problem would already be solved.

In reality, we get trials generated from some sub-optimal policy (often just exploring).

Online Policy Improvement



Online Policy Improvement



Online Policy Improvement

So we get estimates for $q^\pi(s, a)$ for some sub-optimal policy π.

In particular

1. the trial may contain many trash states that should be avoided (instead of wasting time on learning q-values for them)
2. the rewards are mostly useless since the trial does not represent desirable behavior.

Online Policy Improvement

So we get estimates for $q^\pi(s, a)$ for some sub-optimal policy π.

In particular

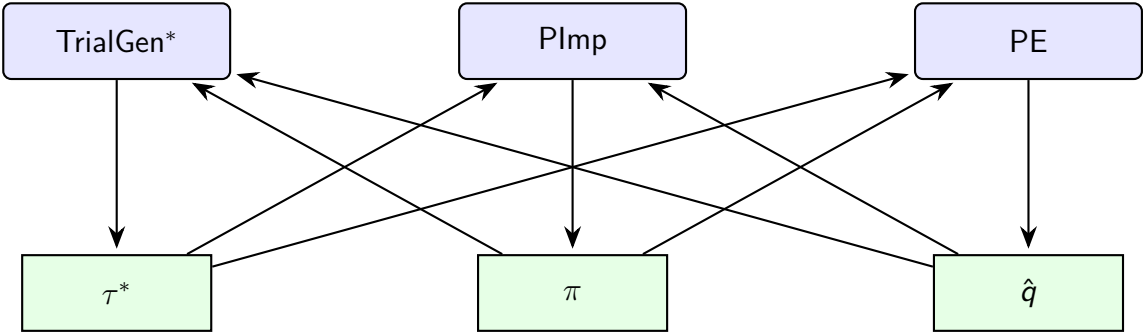
1. the trial may contain many trash states that should be avoided (instead of wasting time on learning q-values for them)
2. the rewards are mostly useless since the trial does not represent desirable behavior.

Q: How can we increase the chance that the trial consists of useful/relevant states?

Motivation for Advanced Policy Improvement

GPI With Centralized Trial Generator

Goal: Create trials that are somewhat optimized, approximating usage of π^*:



Class Overview

1. Motivation for Advanced Policy Improvement
2. Trial-Based Online Policy Improvement
3. Monte-Carlo Tree Search (MCTS)

Trial-Based Online Policy Improvement

Idea: Upgrade trial generation to use a significantly better policy than π.

Trial-Based Online Policy Improvement

Idea: Upgrade trial generation to use a significantly better policy than π.

This seems difficult, since our overall goal is to find such a good policy.

Trial-Based Online Policy Improvement

Idea: Upgrade trial generation to use a significantly better policy than π.

This seems difficult, since our overall goal is to find such a good policy.

Idea of Trial-Based Online Policy Improvement: During trial generation, in state s_t,

- run a mini sub-RL process with limited budget with s_t as initial state and with some exploration technique, trying to approximate only $q(s_t, a)$ for all $a \in A_{s_t}$.

Trial-Based Online Policy Improvement

Idea: Upgrade trial generation to use a significantly better policy than π.

This seems difficult, since our overall goal is to find such a good policy.

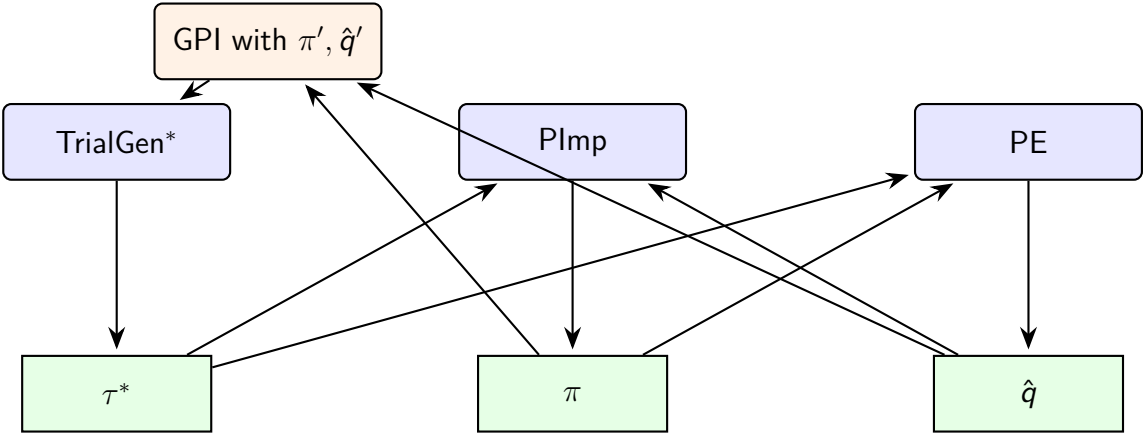
Idea of Trial-Based Online Policy Improvement: During trial generation, in state s_t,

- run a mini sub-RL process with limited budget with s_t as initial state and with some exploration technique, trying to approximate only $q(s_t, a)$ for all $a \in A_{s_t}$.
- then pick each action a with a probability that is proportional to $\hat{q}(s_t, a)$ or number of pulls $N[a]$ obtained from this process (or both).

Trial-Based Online Policy Improvement

GPI With Centralized Trial Generator

Goal: Create trials that are somewhat optimized, approximating usage of π^*:



Trial-Based Online Policy Improvement

We run a new such process for each s_t.

Trial-Based Online Policy Improvement

We run a new such process for each s_t.

In particular:

- at no point is $\hat{q}$ directly updated from $\hat{q}'$.

  Trials in the sub-process are mostly low-quality, and we do not want to contaminate the high-profile $\hat{q}$-values.

Trial-Based Online Policy Improvement

We run a new such process for each s_t.

In particular:

- at no point is $\hat{q}$ directly updated from $\hat{q}'$.

  Trials in the sub-process are mostly low-quality, and we do not want to contaminate the high-profile $\hat{q}$-values.

- we do not share $\hat{q}'$ across different processes.

  However, some inheritance here could be discussed.

Trial-Based Online Policy Improvement

Why is this called *trial-based online policy improvement*?

Trial-Based Online Policy Improvement

Why is this called *trial-based online policy improvement*?

- It is policy improvement, because we enable
  - a better policy through optimized q-values (enables optimized vs greedy decisions)
  - even information about the certainty, by the number of times each action was tried

Trial-Based Online Policy Improvement

Why is this called *trial-based online policy improvement*?

- It is policy improvement, because we enable
  - a better policy through optimized q-values (enables optimized vs greedy decisions)
  - even information about the certainty, by the number of times each action was tried
- It is *online*, because we improve the policy *during* trial generation (and do not even wait for the trial to be completed).

Trial-Based Online Policy Improvement

Why is this called *trial-based online policy improvement*?

- It is policy improvement, because we enable
  - a better policy through optimized q-values (enables optimized vs greedy decisions)
  - even information about the certainty, by the number of times each action was tried
- It is *online*, because we improve the policy *during* trial generation (and do not even wait for the trial to be completed).
- It is *trial-based* because policy improvement is based on a sub-trial process (there are also other forms of online policy improvement).

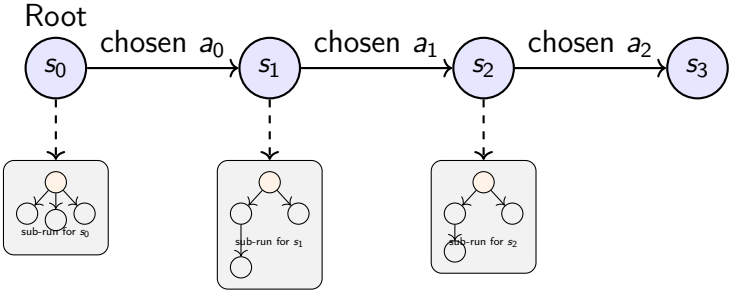
Trial-Based Online Policy Improvement

The fact that we now create trials at two logical levels is often confusing.

We can speak of inner trials (trash trials) and outer trials (optimized trials).

In the context of games, these are often called simulations and episodes/macro-playouts, respectively.

Trial-Based Online Policy Improvement



Each sub-process performs a small local exploration to estimate $\hat{q}(s_t, a)$, selecting a high-quality action for the macro-playout.

Trial-Based Online Policy Improvement

Why are trash inner trials any better than trash trials in the main routine?

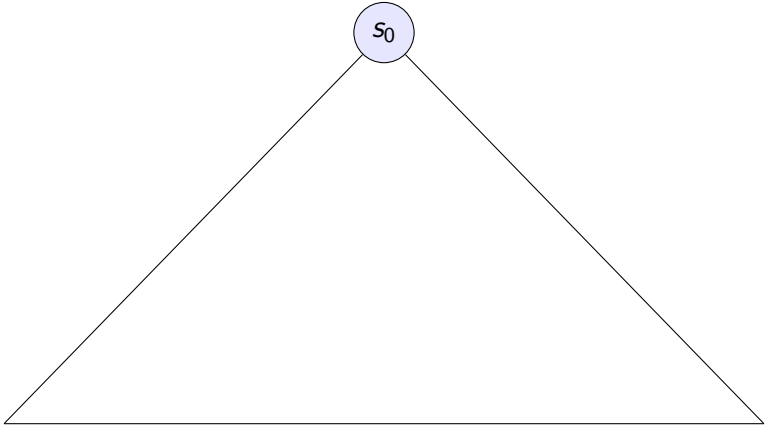
Trial-Based Online Policy Improvement

Why are trash inner trials any better than trash trials in the main routine?

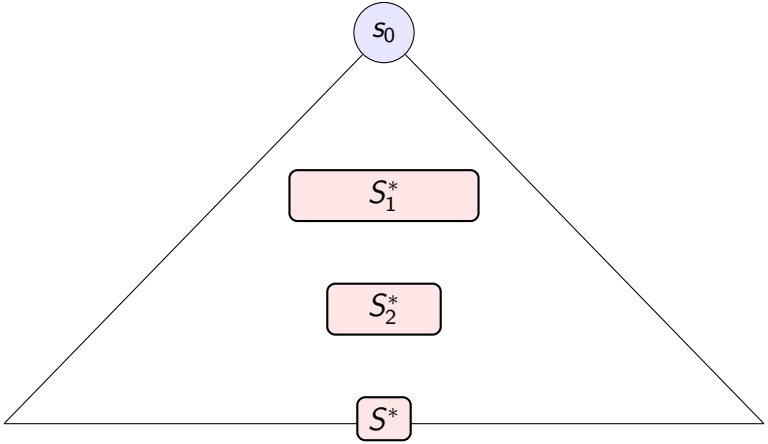
For the following analysis, we assume that the set of desirable states at each depth shrinks exponentially (typical case).

Then we analyze the question from a probabilistic viewpoint.

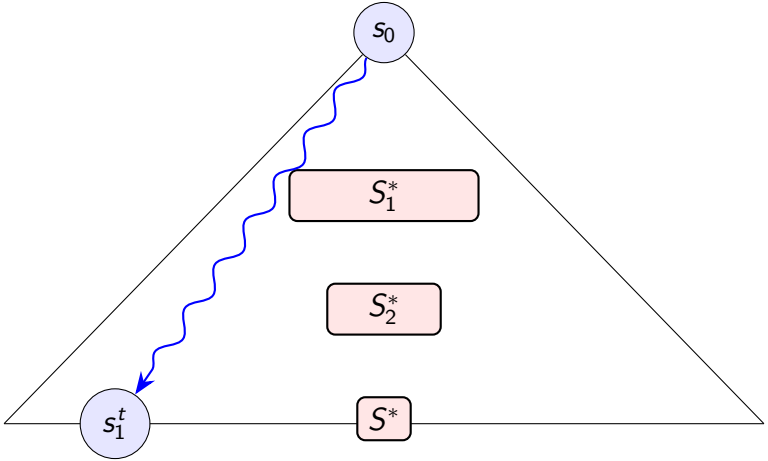
Trial-Based Online Policy Improvement



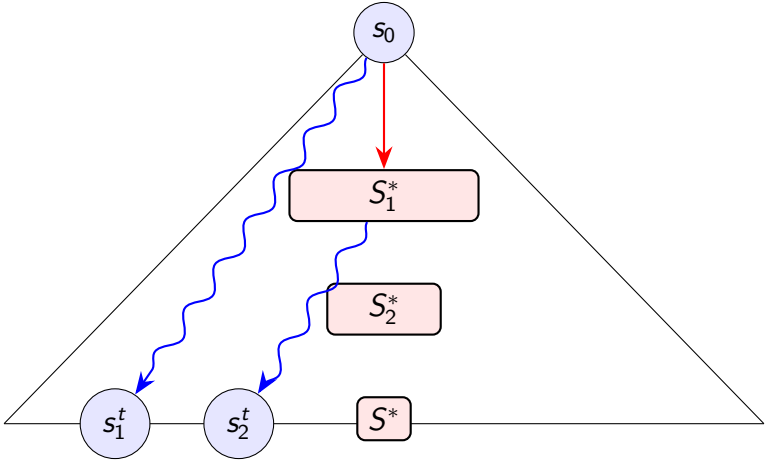
Trial-Based Online Policy Improvement



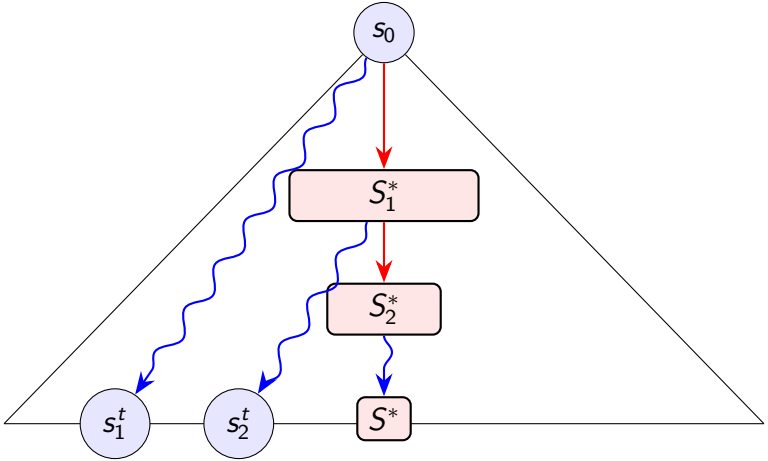
Trial-Based Online Policy Improvement



Trial-Based Online Policy Improvement



Trial-Based Online Policy Improvement



Trial-Based Online Policy Improvement

On a more formal note, suppose that we have a branching factor of 10 and depth 10.

Trial-Based Online Policy Improvement

On a more formal note, suppose that we have a branching factor of 10 and depth 10.

Suppose that $|S^*| = 10$. The probability to hit one of those in at least one out of 10^6 independent runs is

$$1 - \left(\frac{10^{10} - 10}{10^{10}}\right)^{10^6} = 1 - \left(1 - \frac{1}{10^9}\right)^{10^6} = 0.0009995$$

And this implies realizing 10^7 transitions.

Trial-Based Online Policy Improvement

On a more formal note, suppose that we have a branching factor of 10 and depth 10.

Suppose that $|S^*| = 10$. The probability to hit one of those in at least one out of 10^6 independent runs is

$$1 - \left(\frac{10^{10} - 10}{10^{10}}\right)^{10^6} = 1 - \left(1 - \frac{1}{10^9}\right)^{10^6} = 0.0009995$$

And this implies realizing 10^7 transitions.

Even if the runs are not totally independent, the chance of hitting a previously seen state in depth at least 5 is almost 0 (no repetitions, no real learning of $\hat{q}$).

Trial-Based Online Policy Improvement

Now do only 10^5 trials from the root (10^6 transitions), *fix* action a_0, and get a s_1.

Trial-Based Online Policy Improvement

Now do only 10^5 trials from the root (10^6 transitions), *fix* action a_0, and get a s_1.

Since S_1^* is relatively large and we did many trials, it is very likely that $s_1 \in S_1^*$

Trial-Based Online Policy Improvement

Now do only 10^5 trials from the root (10^6 transitions), *fix* action a_0, and get a s_1.

Since S_1^* is relatively large and we did many trials, it is very likely that $s_1 \in S_1^*$

Doing 10^5 trials from s_1 on (only of length 9) implies $9 \cdot 10^5$ transitions.

Trial-Based Online Policy Improvement

Now do only 10^5 trials from the root (10^6 transitions), *fix* action a_0, and get a s_1.

Since S_1^* is relatively large and we did many trials, it is very likely that $s_1 \in S_1^*$

Doing 10^5 trials from s_1 on (only of length 9) implies $9 \cdot 10^5$ transitions.

Even though S_2^* is smaller than S_1^*, the chances are good in ending up there since we start from S_1^*.

Trial-Based Online Policy Improvement

Now do only 10^5 trials from the root (10^6 transitions), *fix* action a_0, and get a s_1.

Since S_1^* is relatively large and we did many trials, it is very likely that $s_1 \in S_1^*$

Doing 10^5 trials from s_1 on (only of length 9) implies $9 \cdot 10^5$ transitions.

Even though S_2^* is smaller than S_1^*, the chances are good in ending up there since we start from S_1^*.

Continuing with this logic, we now only use $10^5(10 + 9 + 8 + 7 + .. + 1) = 5 \cdot 10^6$ transitions in total (half of what we had before) *and* have increased chance to land in S^*.

Trial-Based Online Policy Improvement

Why are trash inner trials any better than trash trials in the main routine?

Trial-Based Online Policy Improvement

Why are trash inner trials any better than trash trials in the main routine?

1. Outer trials enforce exploration in good deep states s_t (inner trials $\neq$ trash trials).

   If every trial starts at s_0, very few trials will pass for s_t, specifically if s_t is far from the root. Outer trials are equivalent to temporarily fixing (good) initial decisions.

Trial-Based Online Policy Improvement

Why are trash inner trials any better than trash trials in the main routine?

1. Outer trials enforce exploration in good deep states s_t (inner trials $\neq$ trash trials).

   If every trial starts at s_0, very few trials will pass for s_t, specifically if s_t is far from the root. Outer trials are equivalent to temporarily fixing (good) initial decisions.

2. The trash-trials are hidden in the sub-routine.

   $\hat{q}'$-updates (playing the idiot) do *not* propagate back and do not contaminate $\hat{q}$ of the main routine (playing the champ); they are forgotten.

Class Overview

1. Motivation for Advanced Policy Improvement
2. Trial-Based Online Policy Improvement
3. Monte-Carlo Tree Search (MCTS)

Monte-Carlo Tree Search (MCTS)

Motivation

Consider a tree MDP with branching factor b.

Suppose that in one of the inner trials we saw a state s_d in depth d.

The probability to see s_d again in

- a new independent inner trial is

Monte-Carlo Tree Search (MCTS)

Motivation

Consider a tree MDP with branching factor b.

Suppose that in one of the inner trials we saw a state s_d in depth d.

The probability to see s_d again in

- a new independent inner trial is $\frac{1}{b^d}$.
- at least one out of t independent trials is

Monte-Carlo Tree Search (MCTS)

Motivation

Consider a tree MDP with branching factor b.

Suppose that in one of the inner trials we saw a state s_d in depth d.

The probability to see s_d again in

- a new independent inner trial is $\frac{1}{b^d}$.
- at least one out of t independent trials is $1 - (1 - \frac{1}{b^d})^t$

Monte-Carlo Tree Search (MCTS)

Motivation

Consider a tree MDP with branching factor b.

Suppose that in one of the inner trials we saw a state s_d in depth d.

The probability to see s_d again in

- a new independent inner trial is $\frac{1}{b^d}$.
- at least one out of t independent trials is $1 - (1 - \frac{1}{b^d})^t$, e.g., for $b = 10$, $t = 1000$
  - 63% if $d = 3$, and
  - 9.5% if $d = 4$

Monte-Carlo Tree Search (MCTS)

Motivation

Consider a tree MDP with branching factor b.

Suppose that in one of the inner trials we saw a state s_d in depth d.

The probability to see s_d again in

- a new independent inner trial is $\frac{1}{b^d}$.
- at least one out of t independent trials is $1 - (1 - \frac{1}{b^d})^t$, e.g., for $b = 10$, $t = 1000$
  - 63% if $d = 3$, and
  - 9.5% if $d = 4$

So $\hat{q}'$-values are only updated/informative if for states extremely close to the root.

Monte-Carlo Tree Search (MCTS)

Idea to increase efficiency: Use and update q-values only for frequent decisions.

Monte-Carlo Tree Search (MCTS)

Idea to increase efficiency: Use and update q-values only for frequent decisions.

Monte-Carlo Tree Search: the GPI process inside of online PImp maintains an explicit tree in which nodes are states from S, and the root is some previously defined state s.

Monte-Carlo Tree Search (MCTS)

Idea to increase efficiency: Use and update q-values only for frequent decisions.

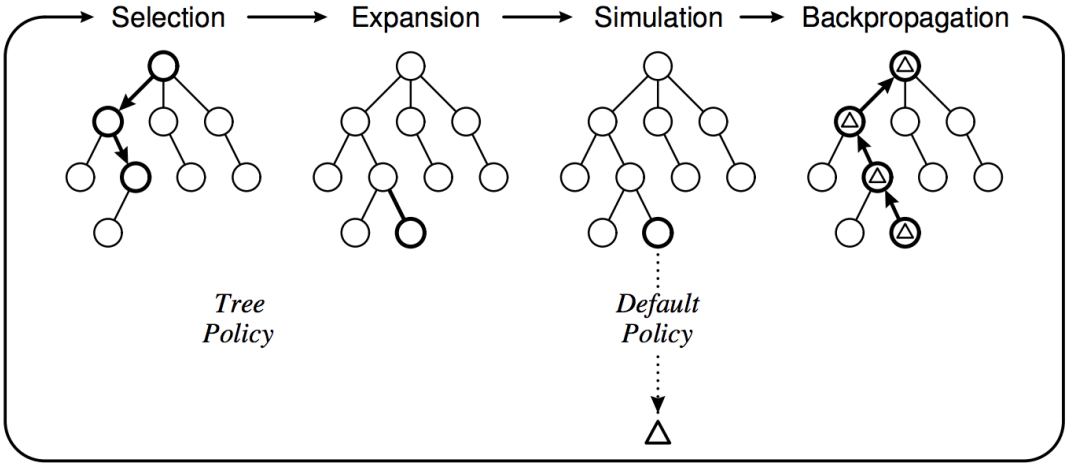
Monte-Carlo Tree Search: the GPI process inside of online PImp maintains an explicit tree in which nodes are states from S, and the root is some previously defined state s.

Speed-Up is achieved by avoiding $\hat{q}'$-access in two ways:

- trial generation only *uses* q-values of frequent states (others just random)
- q-values are only *updated* for frequent states

A state is frequent if it is close to the root.

MCTS


Selection
Expansion
Simulation
Backpropagation
Tree Policy
Default Policy

Monte-Carlo Tree Search (MCTS)

Algorithm

Require: trial_generator, s

Ensure: Approximately optimal policy π^* for s

```
repeat
    use an exploration-exploitation tree policy to reach the last state in the tree
    add border state to the tree
    complete the trial with the default policy
    observe reward r in the leaf
    update statistics for all states on the trial sub-path contained in the tree
until no more episodes
return π, counters N[s][a], and q̂(s, ·)
```

Summary

To summarize the class in one sentence

- standard trial generation has very poor chance to create strong trials if good rewards are sparse,
- online policy improvement dramatically improves the chances to find high quality trials by improving the policy during trial generation,
- Monte-Carlo Tree Search (MCTS) reduces times for these simulations by using and updating only crucial q-values.